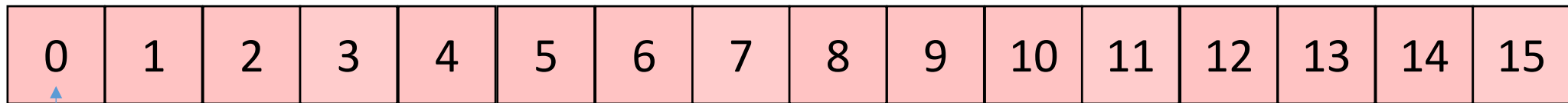


Derived Datatypes

Lecture 17
March 18, 2026

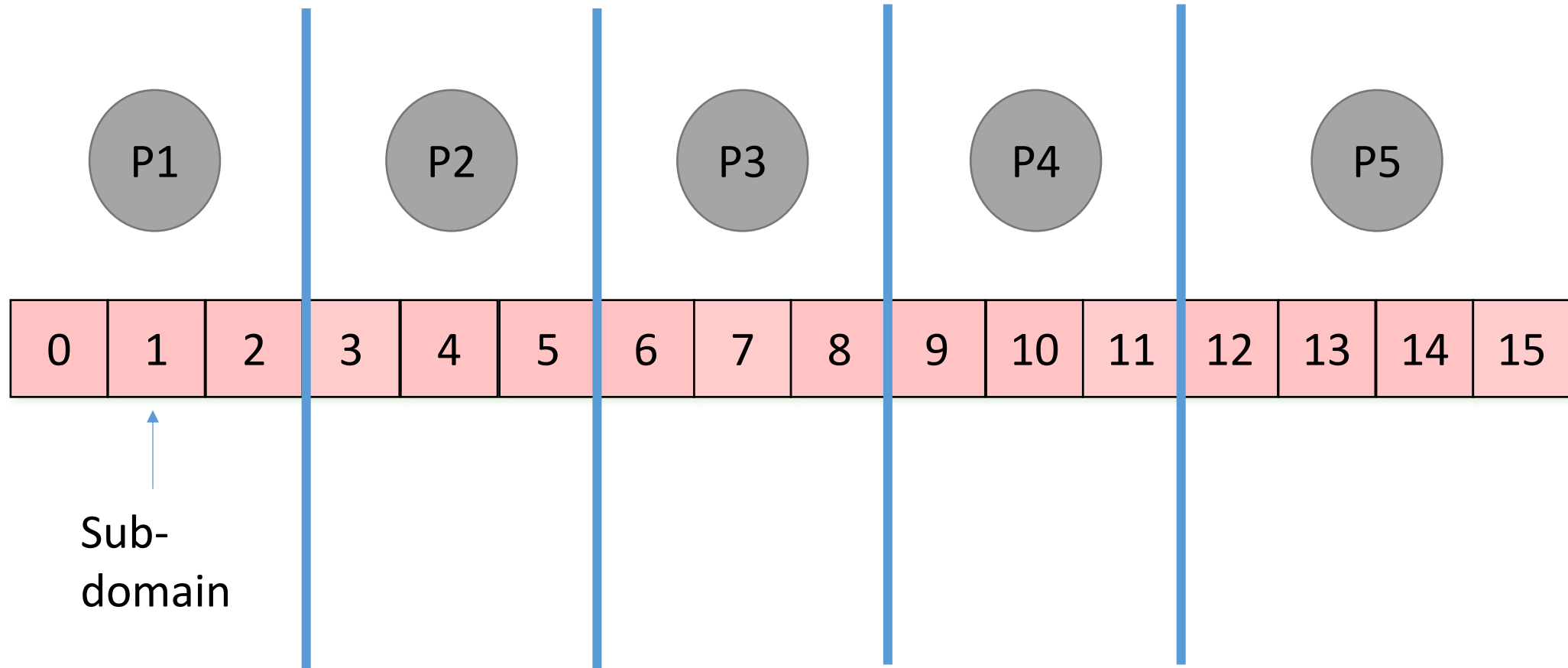
Domain and Domain Decomposition Recap

1D Domain

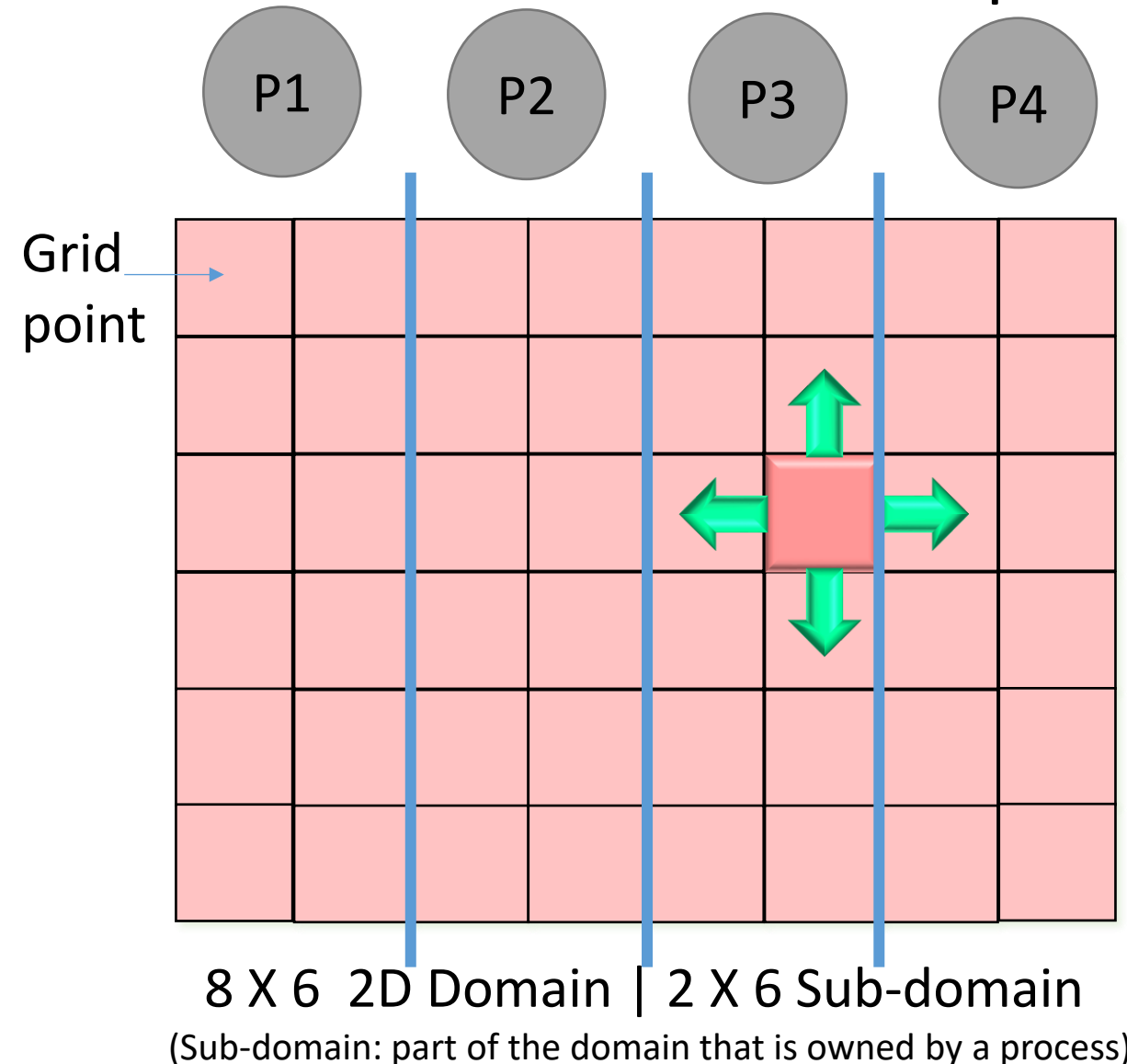


Grid
point

1D Domain Decomposition of 1D Domain



1D Domain decomposition of 2D Domain



Column-wise 1D decomposition

Possible data storage formats in the memory of a process

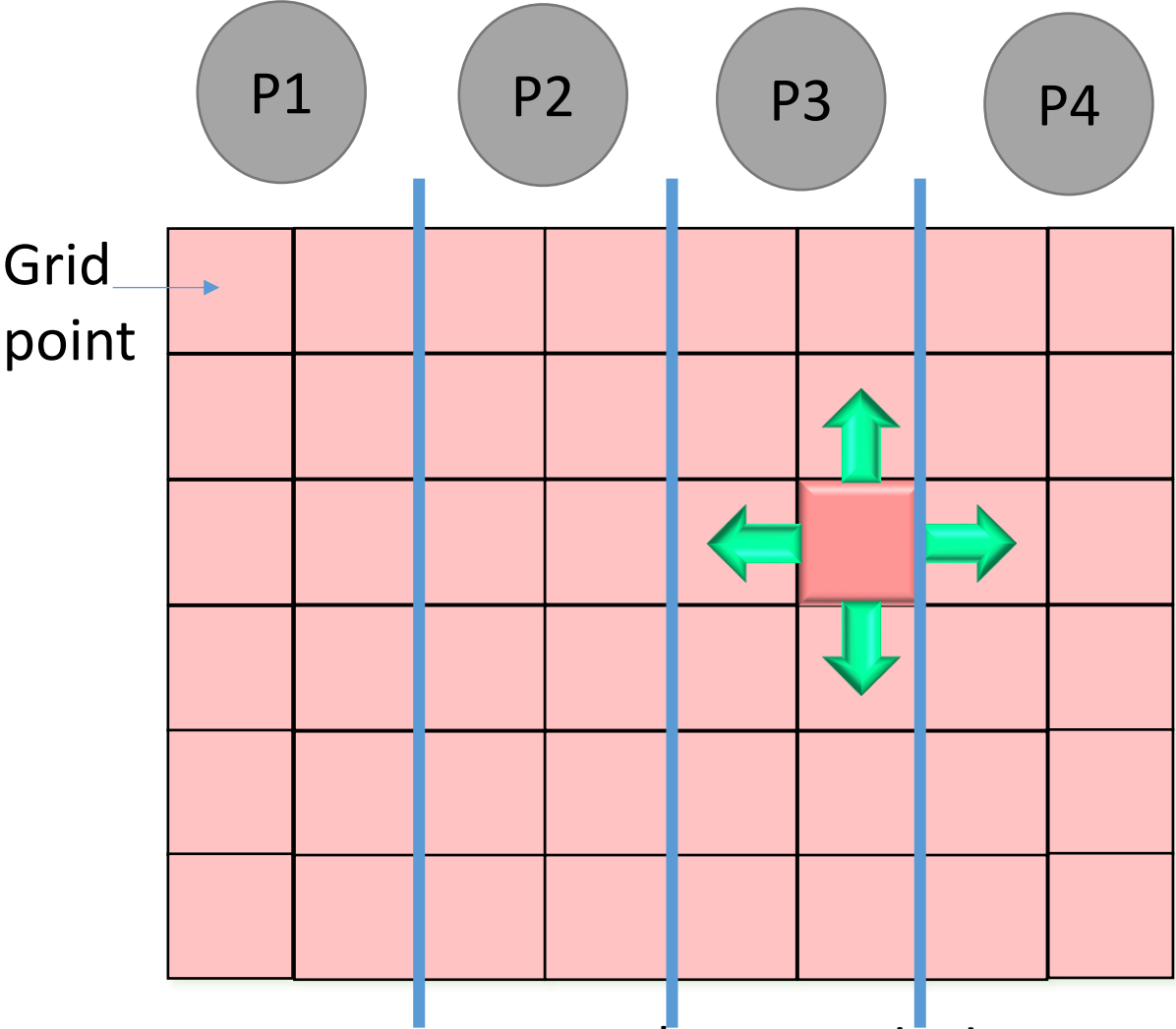
Data[12]
Row-major

Data[0] Data[1]
Data[2] Data[3]
Data[4] Data[5]
...
Data[10] Data[11]

Data[6][2]

Data[0][0] Data[0][1]
Data[1][0] Data[1][1]
Data[2][0] Data[2][1]
...
Data[5][0] Data[5][1]

Read From/Writing To Persistent Storage



8 X 6 2D Domain | 2 X 6 Sub-domain
(Sub-domain: part of the domain that is owned by a process)

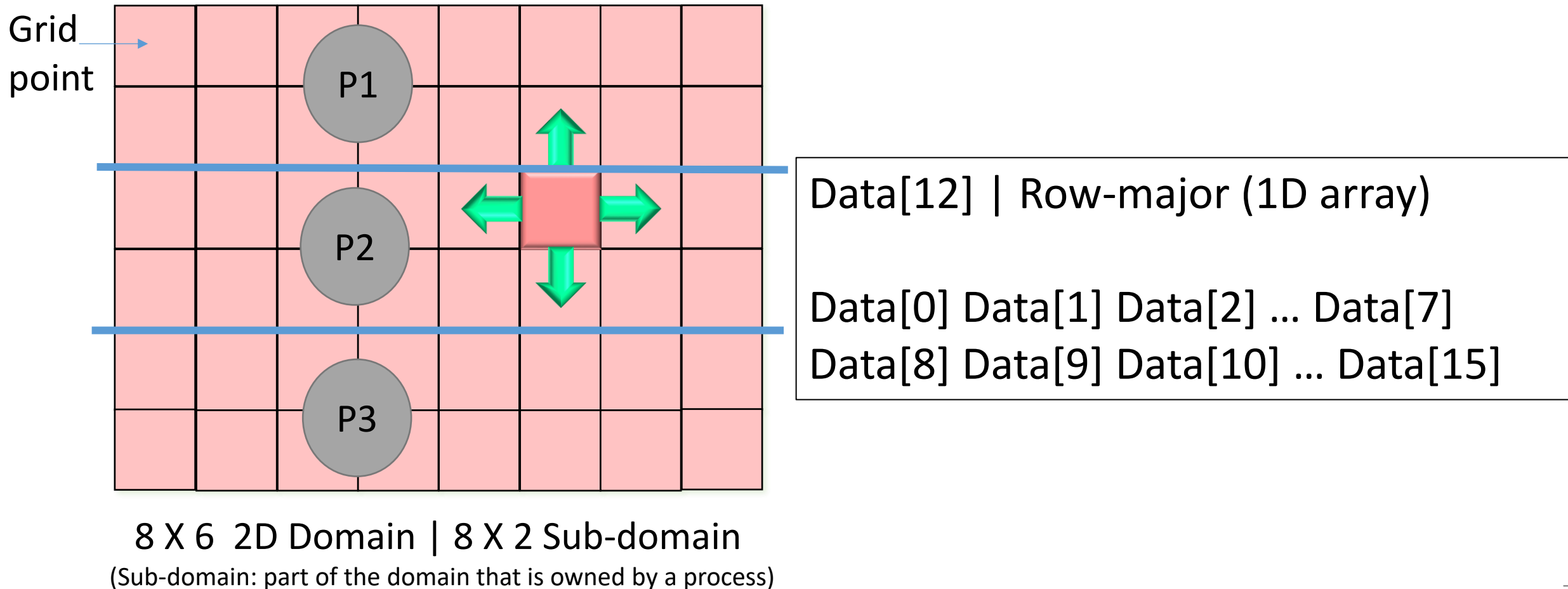
File content: Entire domain in row-major order (sequence of bytes)

Data[0] Data[1]	Data[0] Data[1]	Data[0] Data[1]	Data[0] Data[1]
Data[2] Data[3]	Data[2] Data[3]	Data[2] Data[3]	Data[2] Data[3]
Data[4] Data[5]	Data[4] Data[5]	Data[4] Data[5]	Data[4] Data[5]
...	...	...	...
Data[10] Data[11]	Data[10] Data[11]	Data[10] Data[11]	Data[10] Data[11]

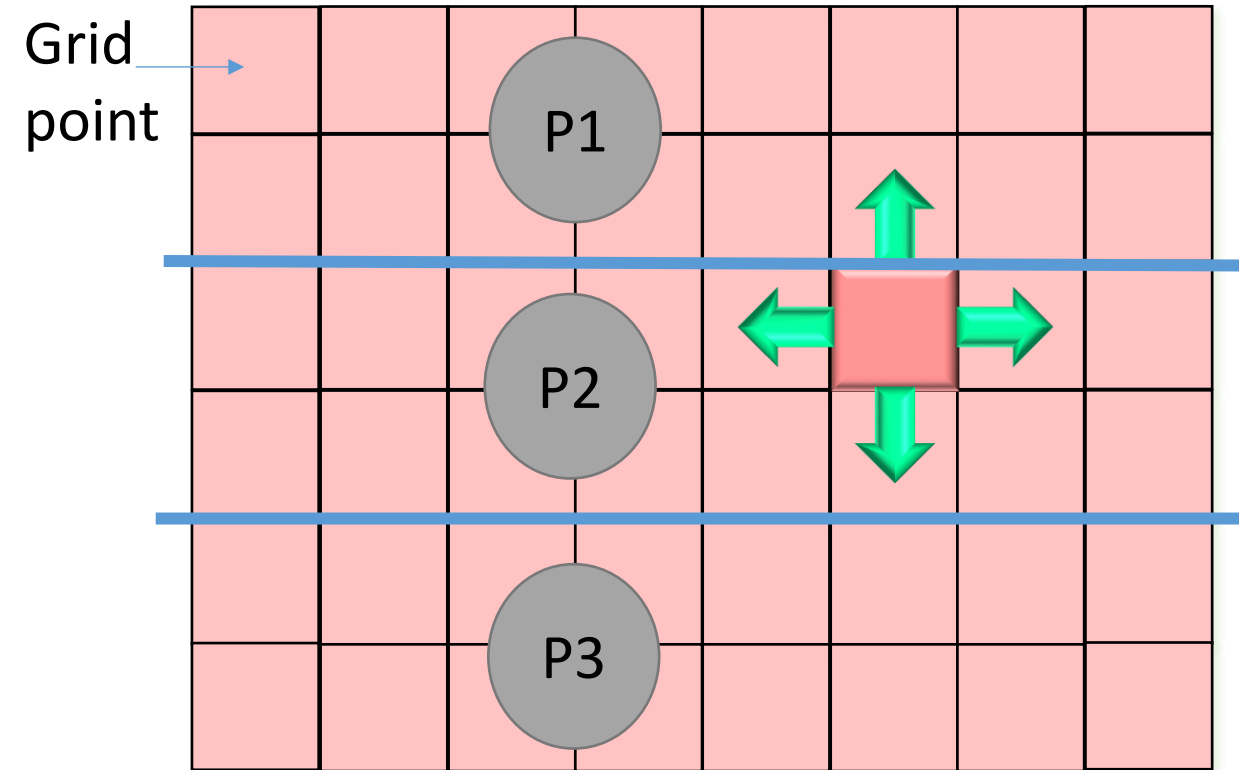
6 rows, 8 columns (for this example)

1D Domain decomposition of 2D Domain

Row-wise 1D decomposition



Read From/Writing To Persistent Storage



8 X 6 2D Domain | 8 X 2 Sub-domain
(Sub-domain: part of the domain that is owned by a process)

Row-wise 1D decomposition

Data distribution depends on decomposition strategy

File content: Entire domain in
row-major order (sequence of bytes)

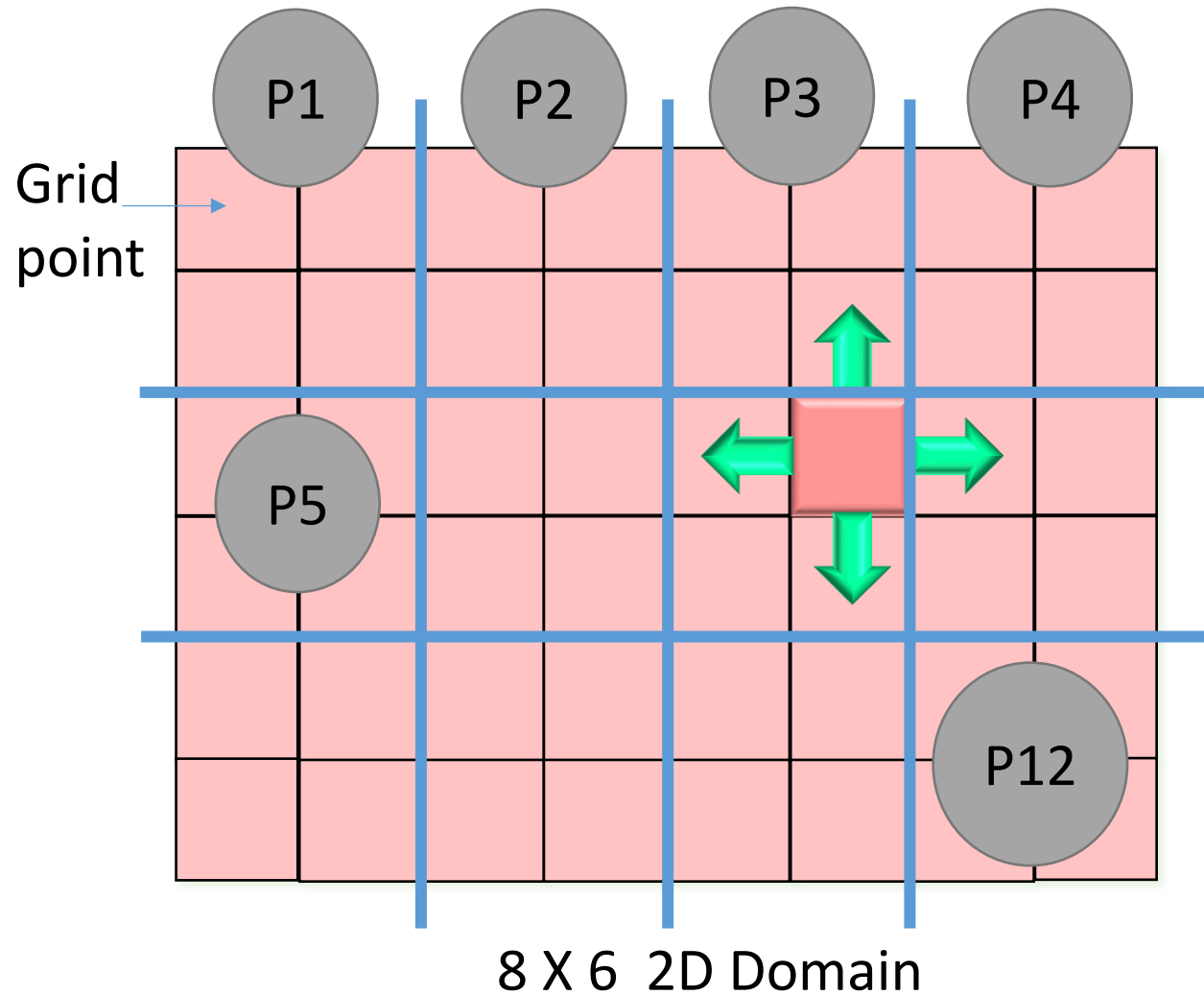
Data[0] Data[1] Data[2] Data[3] Data[4] Data[5] Data[6] Data[7]
Data[8] Data[9] Data[10] Data[11] Data[12] Data[13] Data[14] Data[15]

Data[0] Data[1] Data[2] Data[3] Data[4] Data[5] Data[6] Data[7]
Data[8] Data[9] Data[10] Data[11] Data[12] Data[13] Data[14] Data[15]

Data[0] Data[1] Data[2] Data[3] Data[4] Data[5] Data[6] Data[7]
Data[8] Data[9] Data[10] Data[11] Data[12] Data[13] Data[14] Data[15]

2 rows, 8 columns per process (for this example)

2D Domain decomposition of 2D Domain



Note: Only some process IDs are shown to avoid clutter

XY process decomposition (default)

Possible formats to store data in memory of a process

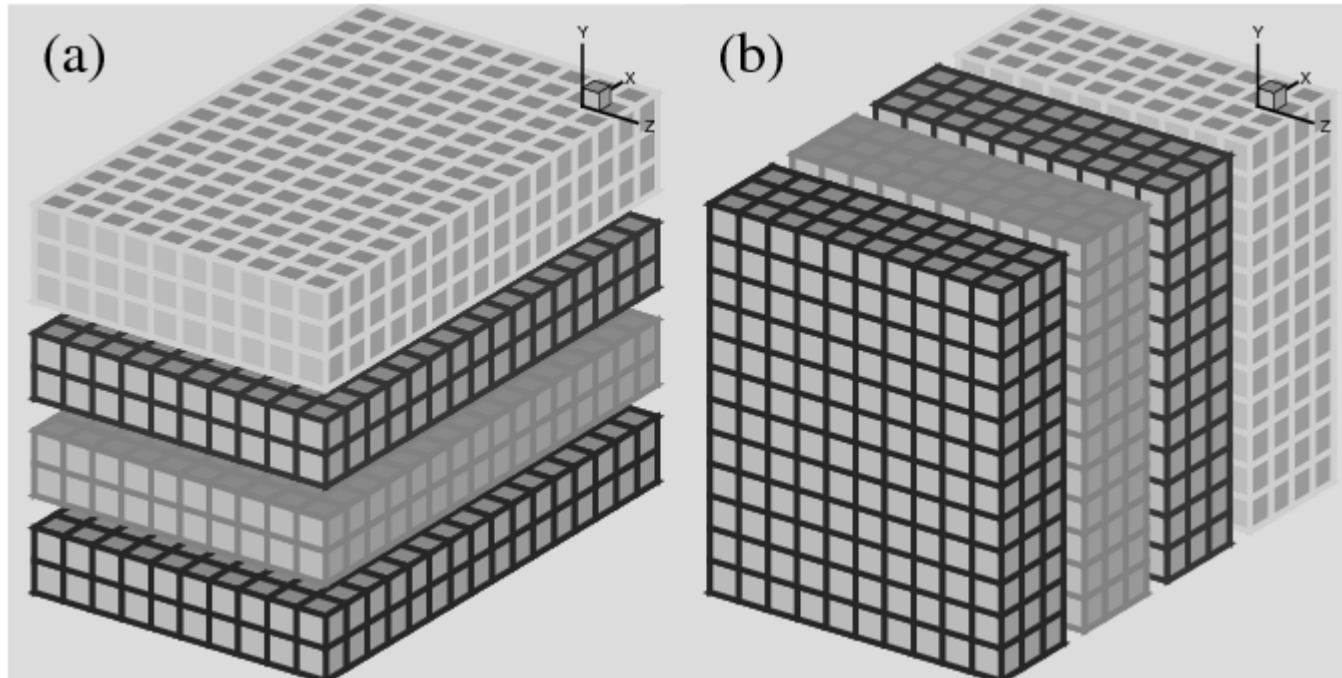
Data[4]
Row-major

Data[0] Data[1]
Data[2] Data[3]

Data[2][2]

Data[0][0] Data[0][1]
Data[1][0] Data[1][1]

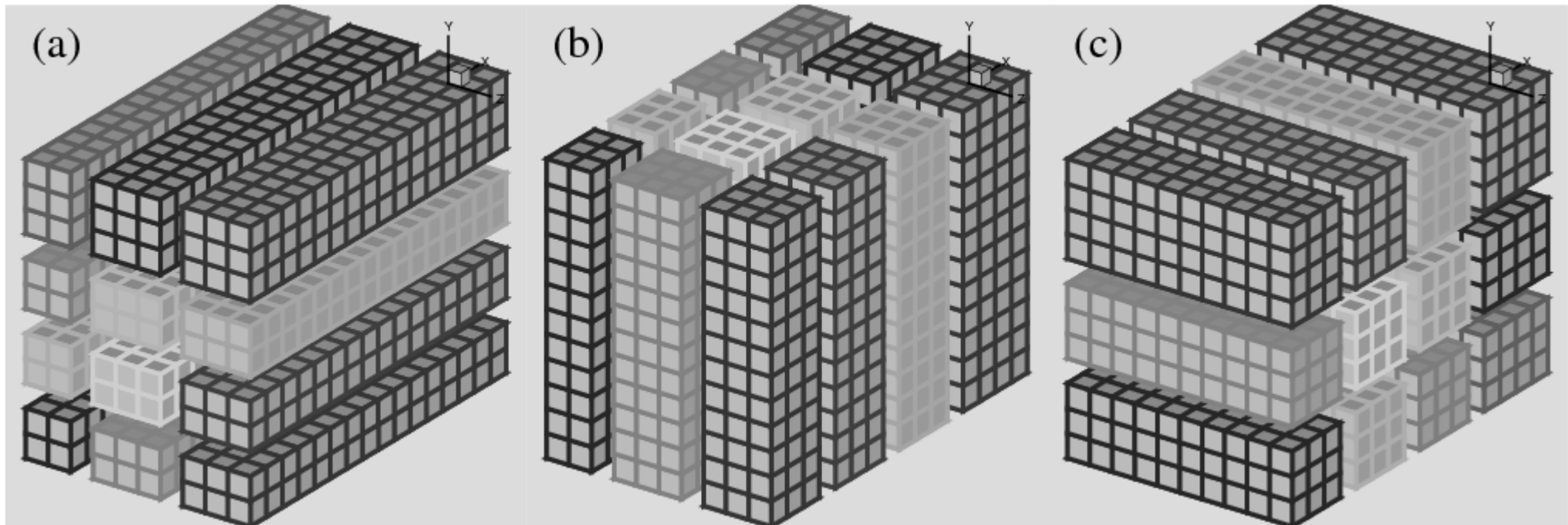
1D Domain Decomposition of 3D Domain



You can also decompose along the third dimension (not shown here)

This is often known as slab decomposition.

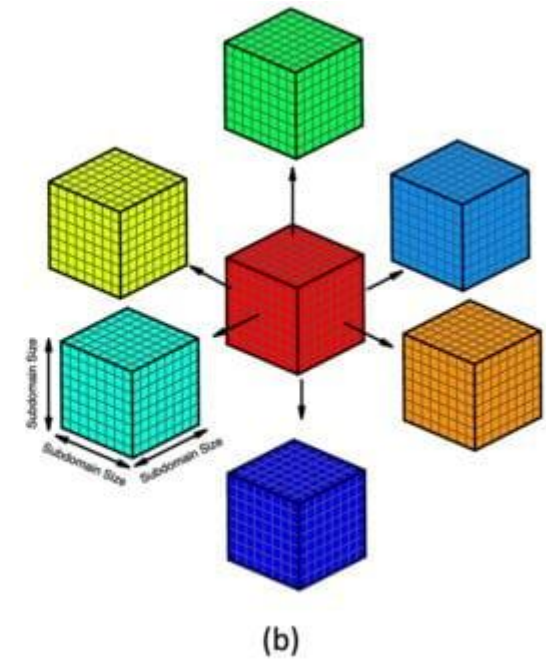
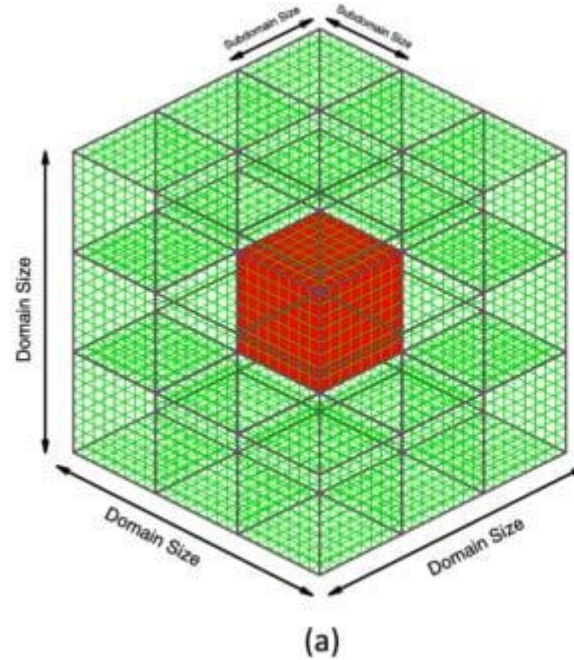
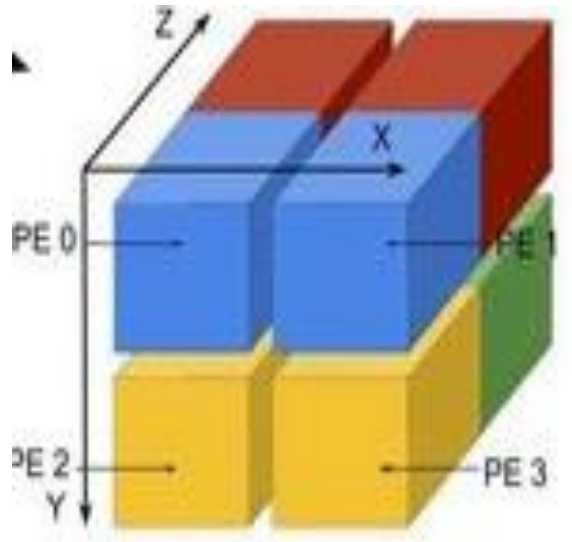
2D Domain Decomposition of 3D Domain



This is often known as pencil decomposition.

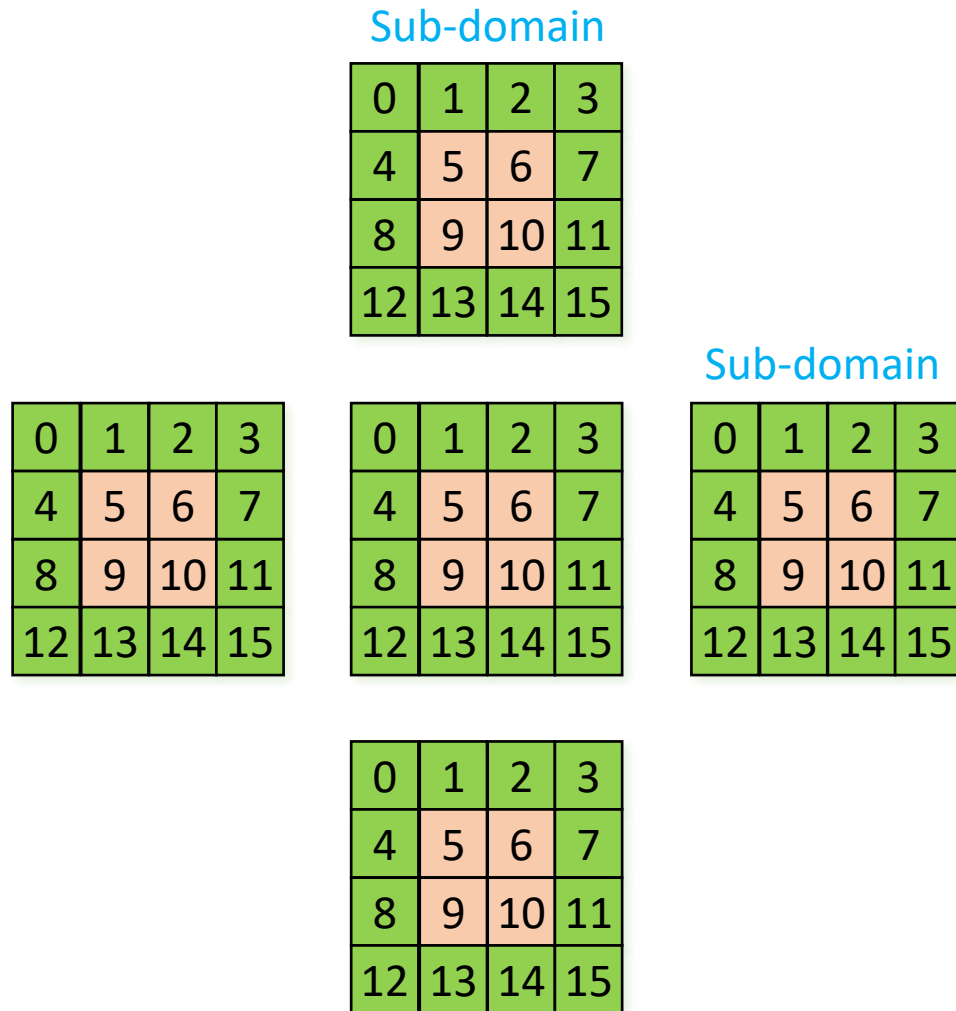
Figure source: hector.ac.uk

3D Domain Decomposition of 3D Domain



Source: <https://www.researchgate.net/>

Halo Exchange



Every simulation time step {

- Stencil computation
 - $Val_{t+1} = \dots$ (may need neighboring points)
- Communicate halo regions
 - Multiple MPI_Sends
 - MPI_Pack + MPI_Send
 - MPI Derived datatype + MPI_Send

}

Note: All subdomains are not shown here

Send / Recv (Option 1)

0	1	2	3				
4	5	6	7				
8	9	10	11				
12	13	14	15				

MPI_Send

MPI_Recv

0	1	2	3	4	5	6	7
---	---	---	---	---	---	---	---

Pack/Unpack and Send / Recv (Option 2)

0	1	2	3				
4	5	6	7				
8	9	10	11				
12	13	14	15				

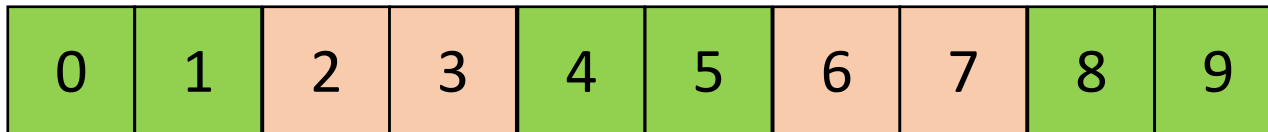
MPI_Pack (buf)
MPI_Send (buf)

MPI_Recv (buf)
MPI_Unpack (buf)

0	1	2	3	4	5	6	7
---	---	---	---	---	---	---	---

Derived Datatypes

MPI_Type_vector



count = #blocks

blocklength = #elements in each block

stride = #elements between the start of each block

count = 3, blocklength = 1, stride = 4

count = 3, blocklength = 2, stride = 4

MPI_Type_vector (count, blocklength, stride, oldtype, newtype)

Code

vector.c

```
int N = atoi (argv[1]);
int count = atoi (argv[2]);
int blocklen = atoi (argv[3]);
int stride = atoi (argv[4]);
int numVectors = atoi (argv[5]);
int data[N];

MPI_Type_vector (count, blocklen, stride, MPI_INT, &newvtype);
MPI_Type_commit (&newvtype);

//initialize data
for (int i=0; i<N; i++)
    data[i]=0;

if (myrank == 0)    /* code for process 0 */
{
    for (int i=0; i<N; i++)
        data[i]=i;
    MPI_Send(data, numVectors, newvtype, 1, 99, MPI_COMM_WORLD);
}
else if (myrank == 1) /* code for process 1 */
{
    printf("\n");
    MPI_Recv(data, numVectors, newvtype, 0, 99, MPI_COMM_WORLD, &status);
    MPI_Get_count (&status, MPI_INT, &recvcount);
    for (int i=0; i<N; i++)
        printf ("%d ", data[i]);
    printf("\n\n");
}

MPI_Type_free (&newvtype);
```

N = Size of array

```
mpirun -np 2
./a.out 10 5 1 2 1
```

```
0 0 2 0 4 0 6 0 8 0
```

```
mpirun -np 2
./a.out 20 5 1 2 2
```

```
0 0 2 0 4 0 6 0 8 9 0
11 0 13 0 15 0 17 0 0
```

Code – Send Selected Columns

vector2D.c

```
int N = atoi (argv[1]);
int column = atoi (argv[2]);
int count = atoi (argv[3]);
int blocklen = atoi (argv[4]);
int stride = atoi (argv[5]);
int data[N][N], received[N*blocklen];

MPI_Type_vector (count, blocklen, stride, MPI_INT, &newvtype);
MPI_Type_commit (&newvtype);

//initialize data
for (int i=0; i<N; i++)
    for (int j=0; j<N; j++)
        data[i][j]=0;

if (myrank == 0)    /* code for process 0 */
{
    for (int i=0; i<N; i++)
        for (int j=0; j<N; j++)
            data[i][j]=column+i+j;
    MPI_Send(
        , 1, newvtype, 1, 99, MPI_COMM_WORLD);
}
else if (myrank == 1) /* code for process 1 */
{
    printf ("\n");
    MPI_Recv(received,
        MPI_INT, 0, 99, MPI_COMM_WORLD, &status);
    for (int i=0; i<count*blocklen; i++)
        printf ("%d ", received[i]);
    printf ("\n\n");
}
```

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

?

?

Code – Send Selected Columns

vector2D.c

```
int N = atoi (argv[1]);
int column = atoi (argv[2]);
int count = atoi (argv[3]);
int blocklen = atoi (argv[4]);
int stride = atoi (argv[5]);
int data[N][N], received[N*blocklen];

MPI_Type_vector (count, blocklen, stride, MPI_INT, &newvtype);
MPI_Type_commit (&newvtype);

//initialize data
for (int i=0; i<N; i++)
    for (int j=0; j<N; j++)
        data[i][j]=0;

if (myrank == 0)    /* code for process 0 */
{
    for (int i=0; i<N; i++)
        for (int j=0; j<N; j++)
            data[i][j]=column+i+j;
    MPI_Send(&data[0][column], 1, newvtype, 1, 99, MPI_COMM_WORLD);
}
else if (myrank == 1) /* code for process 1 */
{
    printf ("\n");
    MPI_Recv(received, count*blocklen, MPI_INT, 0, 99, MPI_COMM_WORLD, &status);
    for (int i=0; i<count*blocklen; i++)
        printf ("%d ", received[i]);
    printf ("\n\n");
}
```

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Examples

```
int N = atoi (argv[1]);  
int column = atoi (argv[2]);  
int count = atoi (argv[3]);  
int blocklen = atoi (argv[4]);  
int stride = atoi (argv[5]);  
int data[N][N], received[N*blocklen];
```

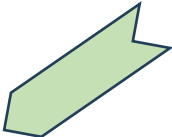
0	1	2	3
1	2	3	4
2	3	4	5
3	4	5	6

```
class $ mpirun -np 2 ./vector2D 4 0 4 1 4
```

```
class $ mpirun -np 2 ./vector2D 4 0 4 2 4
```

Examples

```
int N = atoi (argv[1]);  
int column = atoi (argv[2]);  
int count = atoi (argv[3]);  
int blocklen = atoi (argv[4]);  
int stride = atoi (argv[5]);  
int data[N][N], received[N*blocklen];
```



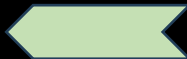
0	1	2	3
1	2	3	4
2	3	4	5
3	4	5	6

```
class $ mpirun -np 2 ./vector2D 4 0 4 1 4
```

```
0 1 2 3
```

```
class $ mpirun -np 2 ./vector2D 4 0 4 2 4
```

```
0 1 1 2 2 3 3 4
```



0	1	2	3
1	2	3	4
2	3	4	5
3	4	5	6

MPI_Type_indexed

0	1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---	---

0	1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---	---

count = #blocks

blocklengths = #elements in each block

displacements = displacement of start of each block

count = 3, blocklengths = 1,2,1 displacements = 0,4,8

count = 3, blocklengths = 2,4,1 displacements = 0,3,8

MPI_Type_indexed (count, blocklengths, displacements, oldtype, newtype)

Code

```
int N = atoi (argv[1]);
int numElements = atoi (argv[2]);    // number of elements to be sent
int count = 3;
int blocklengths[] = {1, 2, 3};
int displacements[] = {0, 3, 6};
int data[N];

MPI_Type_indexed (count, blocklengths, displacements, MPI_INT, &newtype);
MPI_Type_commit (&newtype);

//initialize data
for (int i=0; i<N; i++)
    data[i]=0;

if (myrank == 0)    /* code for process 0 */
{
    for (int i=0; i<N; i++)
        data[i]=i;
    MPI_Send(data,                , newtype, 1, 99, MPI_COMM_WORLD);
}
else if (myrank == 1) /* code for process 1 */
{
    printf("\n");
    MPI_Recv(data,                , newtype, 0, 99, MPI_COMM_WORLD, &status);
    for (int i=0; i<N; i++)
        printf ("%d ", data[i]);
    printf("\n\n");
}
```


Code

```
int N = atoi (argv[1]);
int numElements = atoi (argv[2]);    // number of elements to be sent
int count = 3;
int blocklengths[] = {1, 2, 3};
int displacements[] = {0, 3, 6};
int data[N];
```

```
MPI_Type_indexed (count, blocklengths, displacements, MPI_INT, &newtype);
MPI_Type_commit (&newtype);
```

```
//initialize data
```

```
for (int i=0; i<N; i++)
    data[i]=0;
```

```
if (myrank == 0)    /* code for process 0 */
{
```

```
    for (int i=0; i<N; i++)
        data[i]=i;
```

```
    MPI_Send(data, numElements, newtype, 1, 99, MPI_COMM_WORLD);
```

```
class $ mpirun -np 2 ./indexed 30 1
```

```
    /* code for process 1 */
```

```
    MPI_Recv(data, numElements, newtype, 0, 99, MPI_COMM_WORLD, &status);
```

```
class $ mpirun -np 2 ./indexed 30 2
```

Code

```
int N = atoi (argv[1]);
int numElements = atoi (argv[2]);    // number of elements to be sent
int count = 3;
int blocklengths[] = {1, 2, 3};
int displacements[] = {0, 3, 6};
int data[N];
```

```
MPI_Type_indexed (count, blocklengths, displacements, MPI_INT, &newtype);
MPI_Type_commit (&newtype);
```

```
//initialize data
```

```
for (int i=0; i<N; i++)
    data[i]=0;
```

```
if (myrank == 0)    /* code for process 0 */
{
```

```
for (int i=0; i<N; i++)
    data[i]=i;
```

```
MPI_Send(data, numElements, newtype, 1, 99, MPI_COMM_WORLD);
```

```
class $ mpirun -np 2 ./indexed 30 1
```

0 0 0 3 4 0 6 7 8 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

```
class $ mpirun -np 2 ./indexed 30 2
```

```
0 0 0 3 4 0 6 7 8 9 0 0 12 13 0 15 16 17 0 0 0 0 0 0 0 0
```

Lower Triangular Matrix

```
int N = atoi (argv[1]);
int data[N][N];
int count = N;
int blocklengths[N], displacements[N];
```

```
MPI_Type_indexed (count, blocklengths, displacements, MPI_INT, &newtype);
MPI_Type_commit (&newtype);
```

```
//initialize data
```

```
for (int i=0; i<N; i++)
    for (int j=0; j<N; j++)
        data[i][j]=0;
```

```
if (myrank == 0)    /* code for process 0 */
{
    for (int i=0; i<N; i++)
        for (int j=0; j<N; j++)
            data[i][j]=i+j;
    MPI_Send(data, 1, newtype, 1, 99, MPI_COMM_WORLD);
}
```

mpirun -np 2 ./indexed2D 4

0	1	2	3
1	2	3	4
2	3	4	5
3	4	5	6

Lower Triangular Matrix

```
int N = atoi (argv[1]);
int data[N][N];
int count = N;
int blocklengths[N], displacements[N];

for (int i=0; i<N; i++)
    blocklengths[i] = i+1, displacements[i] = i*N;
MPI_Type_indexed (count, blocklengths, displacements, MPI_INT, &newtype);
MPI_Type_commit (&newtype);

//initialize data
for (int i=0; i<N; i++)
    for (int j=0; j<N; j++)
        data[i][j]=0;

if (myrank == 0)    /* code for process 0 */
{
    for (int i=0; i<N; i++)
        for (int j=0; j<N; j++)
            data[i][j]=i+j;
    MPI_Send(data, 1, newtype, 1, 99, MPI_COMM_WORLD);
}
```

mpirun -np 2 ./indexed2D 4

0	1	2	3
1	2	3	4
2	3	4	5
3	4	5	6

Summary

MPI_Datatype newtype

- MPI_Type_vector (count, blocklength, stride, oldtype, newtype)
- MPI_Type_indexed (count, blocklengths, displacements, oldtype, newtype)

MPI_Type_commit (newtype)

.....

MPI_Type_free (newtype)

Find the E, W, N, S Neighbouring Ranks (2D)

Sub-domain

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Sub-domain

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

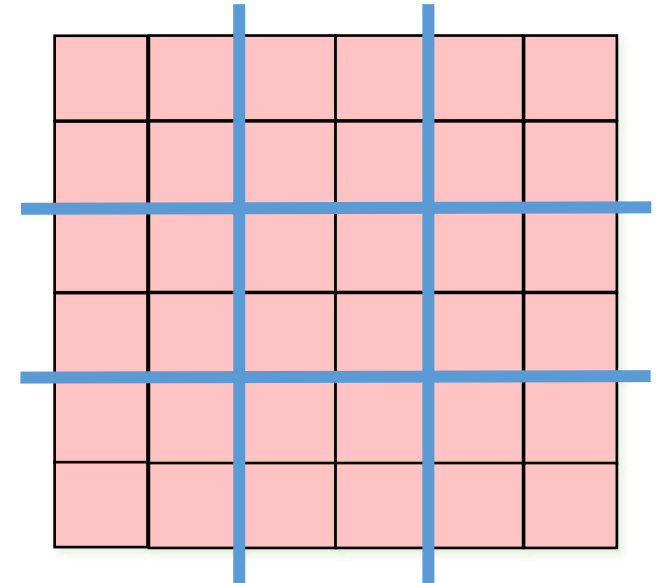
0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

$n_x \times n_y$ data
domain

$p_x \times p_y$
processes



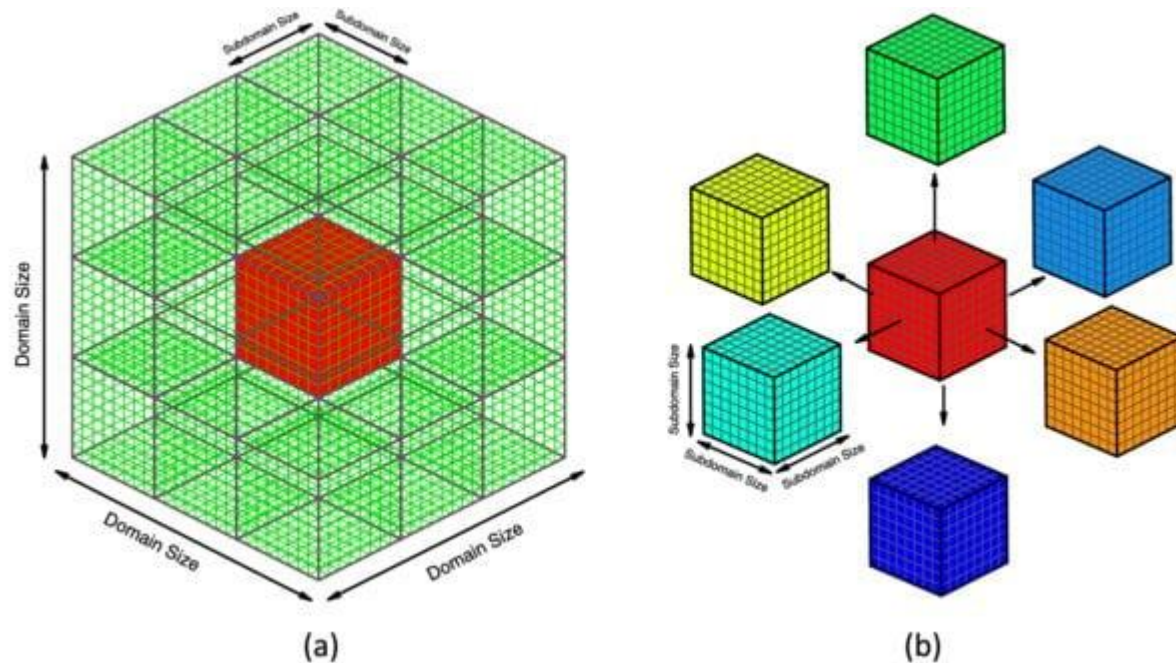
E (right)
W (left)
N (up)
S (down)

$\text{myrank}+1$
 $\text{myrank}-1$
 $\text{myrank}-p_x$
 $\text{myrank}+p_x$

H.W.:
Complete...
Missing
conditions

Note: All subdomains are not shown here

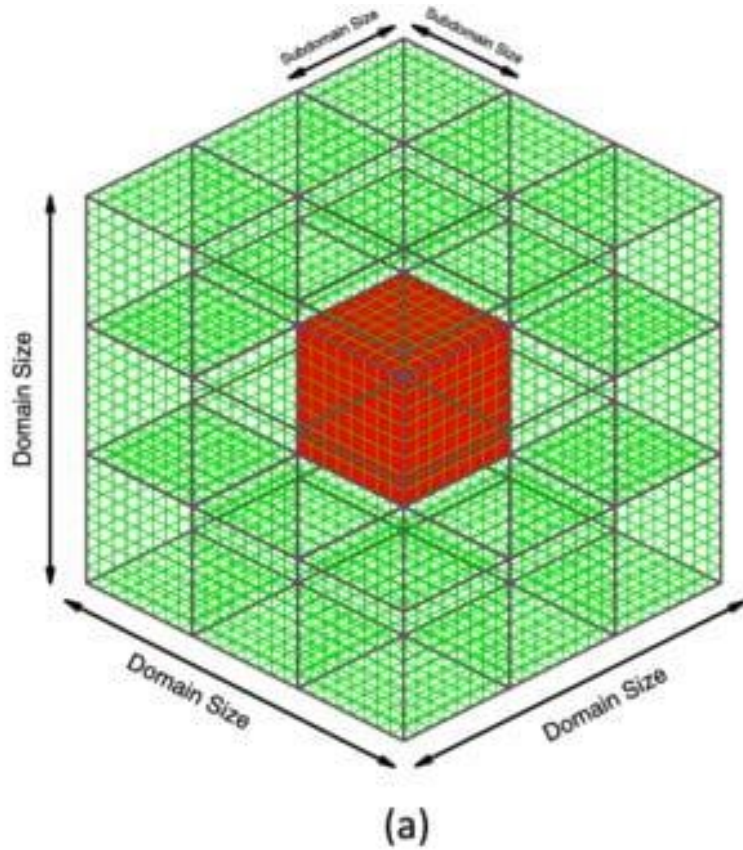
3D Domain Decomposition of 3D Domain: Find the neighbouring ranks



H.W.:
Complete...

Source: <https://www.researchgate.net/>

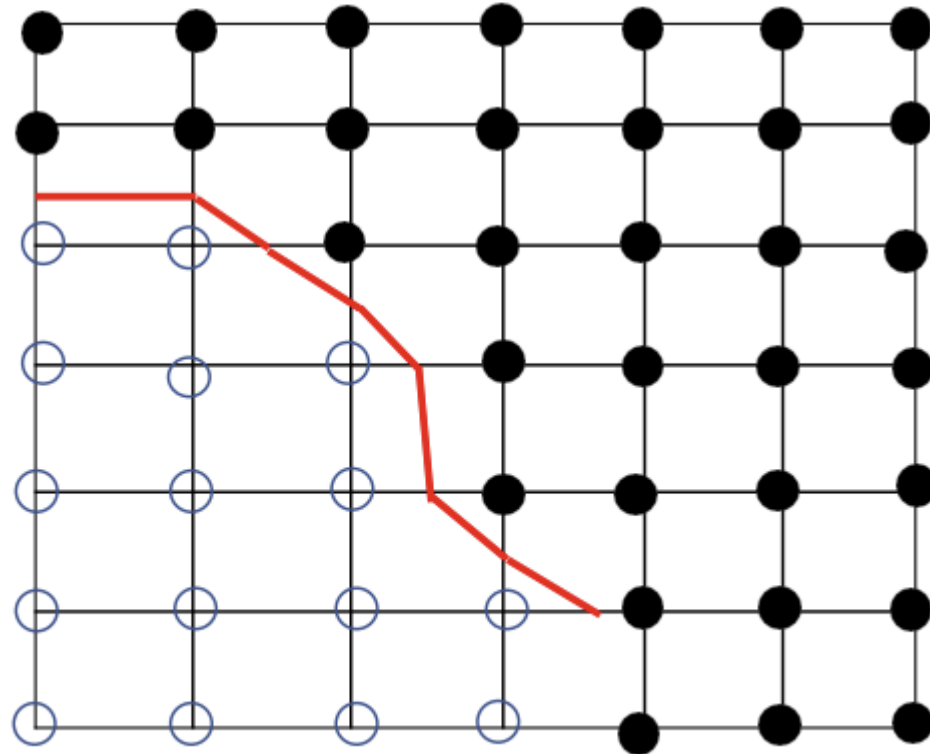
Assignment 2



- A number of data fields (variables) defined at each grid point of a 3D domain
- Halo exchange
- Compute d-point stencil
- Compute the count of local isovalues (note that you do not have to extract the isosurface)

2D Isocontour Extraction: Marching Squares

- Given a 2D scalar field, compute isocontour (isoline) for isovalue = **C**
- This is usually done in a cell-by-cell manner using Marching Squares algorithm



Contour value (isovalue) = C

● Value $> C$

○ Value $< C$

Source: CS677: Topics in Large Data
Analysis And Visualization